USAGE GUIDE FOR

PMINI FLASHCARD

PRODUCT

HTTPS://SHOP.GILTESA.COM/PRODUCT/POKEMON-MINI-FLASHCARD

INDEX

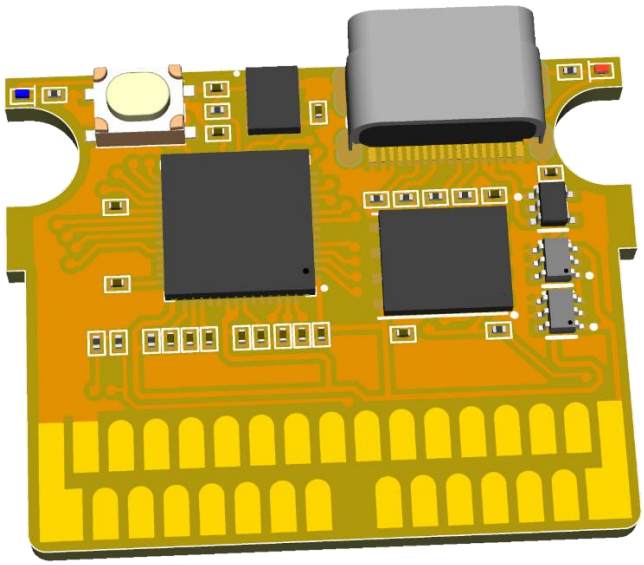
DESCRIPTION

Flash cartridge for the Pokémon Mini console, built around the **Raspberry Pi RP2040** microcontroller platform. **It includes 16 MB of onboard memory** to load and run games, along with a **USB-C interface** for convenient firmware updates and game transfers.

Whether you're developing, testing, or simply playing, this cartridge offers a reliable and space-efficient solution for getting the most out of your Pokémon Mini.

FEATURES

- Designed for the Pokémon Mini console.
- Powered by the RP2040 dual-core microcontroller.
- 16 MB onboard flash memory for storing games:
- More than enough to store multiple game backups, thanks to the small size of official titles and homebrew.
- USB-C interface for easy programming and firmware updates.
- Compact and low-profile design optimized for limited space.
- Programable status LED and USB-C power LED.
- Ideal for homebrew development, testing, and game use.



INCLUDED

- Pokémon Mini flash cartridge (fully assembled).

REQUIRED

- Pokémon Mini console.
- USB-C cable (for programming and game uploading).
- Computer with a web browser (for flashing games).

NOTES

- No ROMs are provided. Intended for personal backups and homebrew (available at pokemon-mini.net).

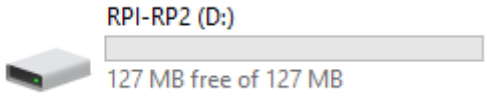
ATTRIBUTIONS

- This product was designed entirely from scratch, but it was inspired by the PM2040 project by zwenergy, who created an adapter for soldering a Waveshare RP2040 Zero, along with software tools for flashing the microcontroller. It intentionally uses the same pinout to ensure compatibility with his software.
  Many thanks to zwenergy for his excellent work! His code was used as a base and adapted for this cartridge.
- Menu font by Stephen Denne.
- Menu and ROM Patcher improvements assisted by GPT.
- Circuit logo image by Fidel Castro.
- Font generated using FontMeme.
- Pikachu surfing image by dragon.
- Pokeball image by seabranddesign.

PRODUCT DETAILS

The cartridge features a USB-C connector that allows connection to a computer to load new firmware.

To update the firmware, hold down the **FLASH** button on the cartridge while connecting the USB-C cable. This will place the cartridge in **flash mode**, and it will appear on the computer as a **storage drive**.



It is not possible to copy files or games directly to the cartridge; only **firmware files** with extension **.uf2** are supported. If the cartridge is connected to the computer without pressing the **FLASH** button, nothing will happen by default. However, a firmware developed by **zwenergy** can be used to create **backups of saved games**.

OPENING THE CARTRIDGE

It is completely unnecessary to open the cartridge, but if you still want to do it, take special care. The top and bottom casing are not only held together by the two visible screws but also by two small internal tabs.

You should carefully pry at the bottom of the casing, as shown in the following image, to release the tab. Do this on both sides.

PHOTO

FIRMWARE GENERATION

To load games onto the console, it is necessary to generate a firmware that includes them. Once generated, it can be copied to the cartridge by simply dragging it to the storage drive that appears on the computer (only while holding the FLASH button during the USB-C connection).

Firmware generation is a complex process. To simplify it when only changing the games, the Web ROM Patcher utility is available.

If you wish to create a firmware from scratch, the process is explained later in this guide. However, this is only useful if you want to customize the behavior of the firmware or the menu. For most users, the Web ROM Patcher utility will cover all their needs.

WEB ROM PATCHER

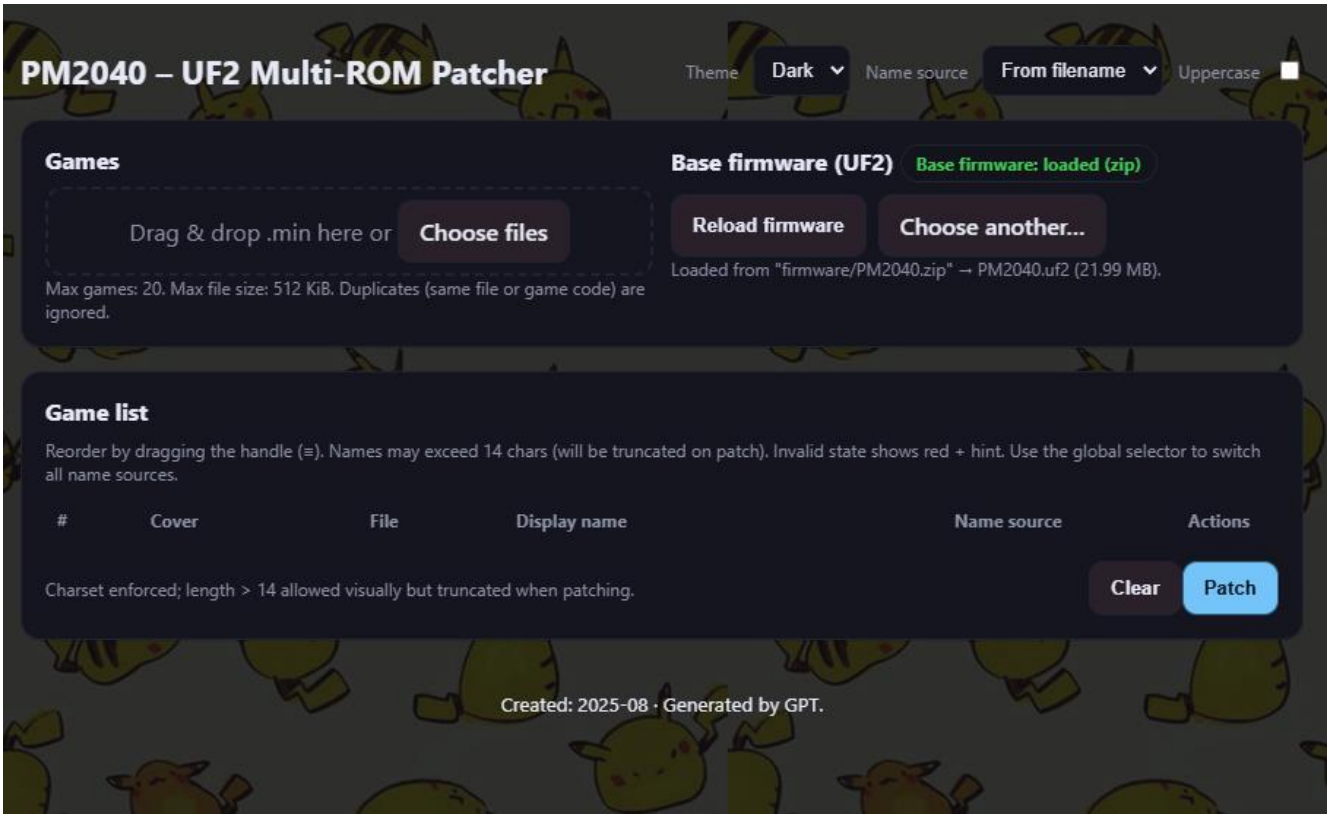
You can access this web utility at the following address:

https://shop.giltesa.com/pm
https://giltesa.github.io/Pmini-Flashcard-Code

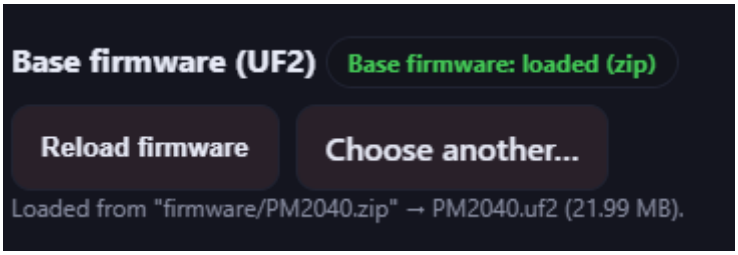
The utility runs as a web page and can also be used locally. If desired, you can download the complete page from the repository:

https://github.com/giltesa/Pmini-Flashcard-Code/tree/main/4.%20ROM%20Patcher

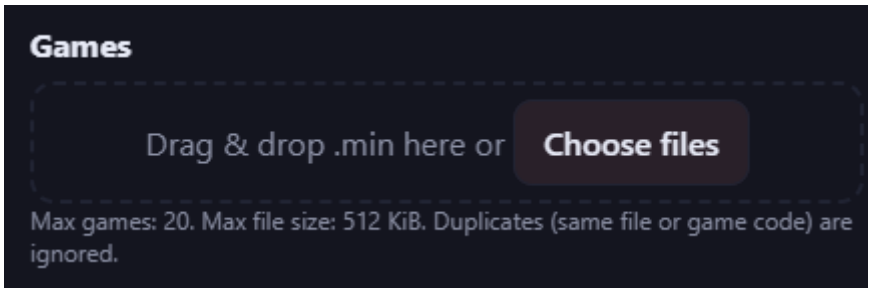
When you open the web page, you will see an interface similar to the one shown in the following screenshot:



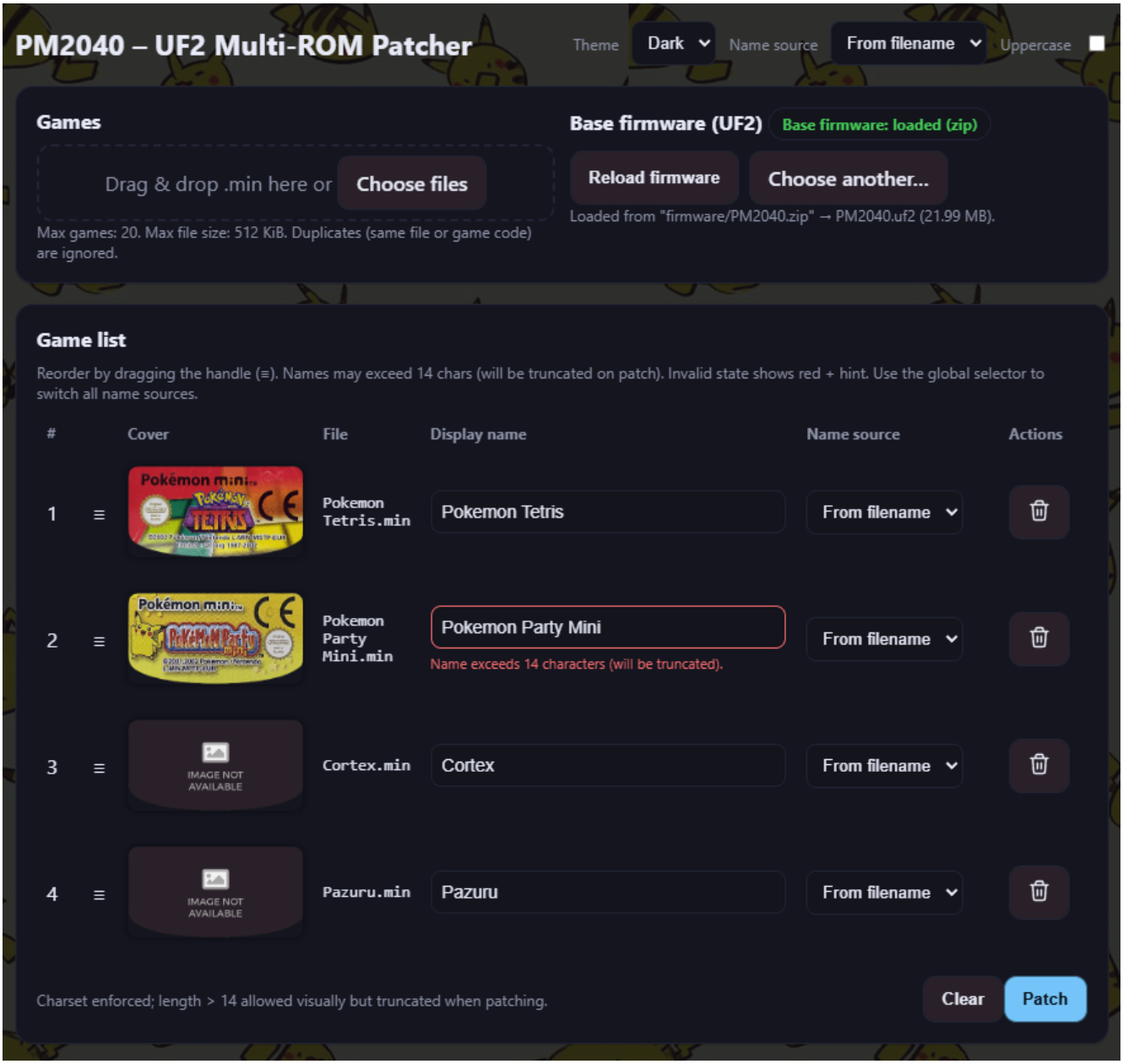
When the web page is loaded, it will use the default base firmware. Optionally, you can select your own firmware.



Next, you can select or drag the games you want into this section:

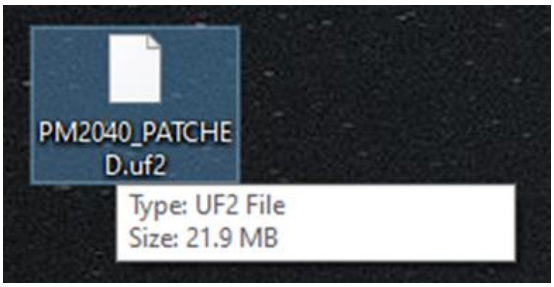


This will add each game to the game list. From this list, you can change the name that will appear in the game selection menu and adjust the order in which the games are displayed in the menu.



By default, the file name is used. You can also use the game name or enter any name manually. In all cases, the name is limited to 14 standard characters (ASCII).

Finally, click the **Patch button**, which will generate a new **PM2040_PATCHED.uf2** that can be written to the cartridge.



The writing process takes some time, as the firmware is designed to support up to 20 games. Once the firmware has been written, the cartridge will automatically disconnect from the computer and will be ready for use on the console.

FIRMWARE GENERATION FROM SCRATCH

The compiled firmware is a file with the **.uf2** extension. To flash it onto the **PMini FlashCard**, connect the cartridge to the computer via USB while holding down the **FLASH** button. This will make it appear as a storage drive, but only the **.uf2** file can be copied onto it. Games cannot be dragged directly into the drive.

When creating the firmware manually, it can either contain a single game or include a special game that serves as a menu, allowing the user to select which game to run from a list.

Regardless of whether the firmware is single-ROM or multi-ROM, the operation is the same: it emulates the cartridge memory so the console can read game data. In single-ROM firmware, it only needs to send the data of that one game for the console to run it. However, with multi-ROM firmware, things get more complex.

The multi-ROM firmware consists of two data banks *(arrays of type uint8_t)*. One bank contains the menu, which is simply a game developed for the Pokémon Mini using the c88-pokemini development tools. This menu is programmed to read games from specific memory addresses. The second data bank is where those games are stored; within the firmware, this area is initially filled with the placeholder 'R','O','M','S','T','A','R','T' followed by zeros (empty space) until the full allocated size is reached. Later, the WebApp patches the games into the firmware by locating that placeholder, which matches the memory position expected by the menu to read the games.

By design, the game data bank always has a specific capacity that is a multiple of 512KB, which is the maximum size of an official game. For example, 1024KB for two games, 1536KB for three, and so on, limited by the RP2040's 16MB memory used in the **PMini FlashCard**, as well as by the menu's ability to display and select each game through its interface.

Each game slot always has the same capacity, so if smaller homebrew games are used, the unused space is simply wasted. While not ideal, this is not a serious issue since the total number of games and homebrew titles is relatively small, and the cartridge's overall capacity is quite large.

CODE

The code for the PMini FlashCard repository is composed of several repositories, both from **zwenergy**:

- The first repository contains the main code, including the firmware and the WebApp:
  http://github.com/zwenergy/PM2040
- The second repository contains the game selection menu:
  https://github.com/zwenergy/PM2040_MultiMenu

1. TOOLS FOR COMPILING THE FIRMWARE

Both for compiling the firmware and for the menu, a large number of programs are required. Below are the ones related to the firmware.

PYTHON

It is used to run scripts that generate the data files needed to compile the game.

- Download from https://www.python.org/downloads
- Install it; in my case, the file: `python-3.13.5-amd64.exe`
- Verify that it installed correctly: open a new command prompt and type `python --version`, which should return something like `Python 3.8.2`

CMAKE

It is used to generate the configuration files necessary to compile the code.

- Download from https://cmake.org/download
- Install it; in my case, the file: `cmake-4.1.0-rc4-windows-x86_64.exe`
- Add the installation directory to the Windows environment variables PATH; in my case: `C:\Program Files\CMake\bin`
- Verify that it is correctly installed and that the PATH is configured properly: open a new command prompt and type `cmake --version`, which should return `cmake version 4.1.0-rc4`

NINJA

It is used as a fast and efficient build system that, together with CMake, executes the compilation tasks defined to generate the firmware.

- Download from https://github.com/ninja-build/ninja/releases
- Extract and save it in the desired directory
- Add its path to the Windows environment variables; in my case: `C:\Users\giltesa\Desktop\ninja`
- Verify that it is accessible from the command prompt: open a new command prompt and type `ninja --version`, which should return `1.13.1`

ARM GNU TOOLCHAIN

It is used to compile the source code and generate executable files compatible with the console's ARM processor.

- Download from https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads
- Install it; in my case, the file: ***`arm-gnu-toolchain-14.3.rel1-mingw-w64-x86_64-arm-none-eabi.exe`***
- Add its path to the Windows environment variables; in my case: ***`C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.3 rel1\bin`***
- Verify that it is installed correctly and that the PATH is properly configured: open a new command prompt and type ***`arm-none-eabi-gcc --version`***, which should return ***`arm-none-eabi-gcc (Arm GNU Toolchain 14.3.Rel1 (Build arm-14.174)) 14.3.1 20250623`***

GIT

It is mainly used to download the source code and required files from the official repository.
It is optional, as the files can be downloaded manually from the website, but using Git is more convenient.

- Download from https://git-scm.com/download/win
- Install it; in my case, the file: ***`Git-2.50.1-64-bit`***
- Verify that it is installed correctly: open a new command prompt and type ***`git --version`***, which should return ***`git version 2.50.1.windows.1`***

PICO SDK

It is used to develop and compile programs intended to run on the RP2040 microcontroller.

- All of this is done from the command prompt
- Choose where you want to have this tool and move to that directory, for example: ***`cd C:\Users\giltesa\Desktop`***
- Download the tool with ***`git clone https://github.com/raspberrypi/pico-sdk.git`***
- Create a new Windows user environment variable named ***`PICO_SDK_PATH`*** with the value pointing to the download path selected earlier; in my case: ***`C:\Users\giltesa\Desktop\pico-sdk`***

FIRMWARE SOURCE CODE

- Decide where you want to store the firmware code and navigate to that directory
- For example: ***`cd C:\Users\giltesa\Desktop`***
- Download the code with ***`git clone https://github.com/giltesa/Pmini-Flashcard-Code.git`***

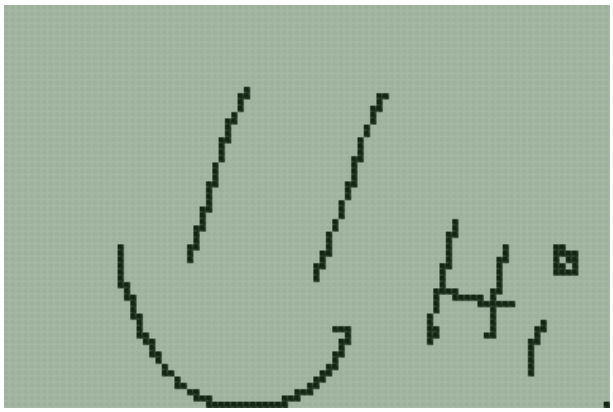
2. COMPILING THE FIRMWARE

If all the tools installed and configured so far are set up correctly, it is time to create the firmware.

- Open the command prompt and navigate to the code directory. In my case: `cd C:\Users\giltesa\Desktop\Pmini-Flashcard-Code\1. Firmware`
- Create the build directory: `mkdir build`
- Move into that directory: `cd build`
- Run the command: `cmake -G "Ninja" ..`
- Compile the project with: `ninja`

The last two commands will generate many files in the build directory, including PM2040.uf2, which is the single-ROM firmware containing the Hello World homebrew. Drag it to the cartridge's storage drive, and once the writing process is complete, disconnect the USB-C cable and insert the cartridge into the console.

You should see the following on the screen:



3. TOOLS FOR COMPILING THE MENU

For the menu, fewer tools need to be installed since some are already common, and those steps are no longer necessary.

C88-POKEMINI

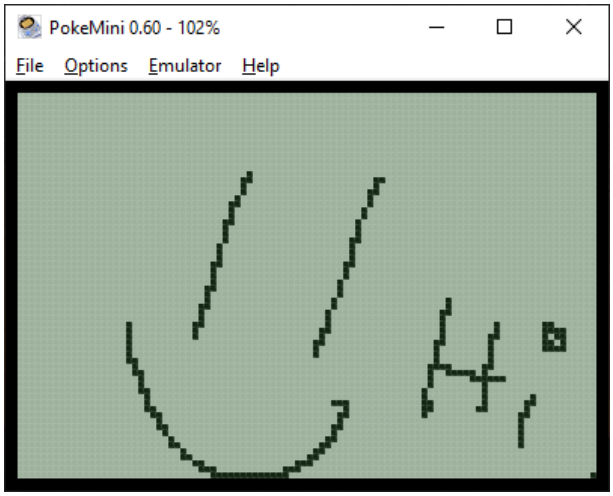
It is used to compile the source code and generate executables specifically for the Pokémon Mini console.

- Open the command prompt and navigate to the directory where you want to have this tool (make sure there are no spaces in the path). For example: `cd C:\`
- Download the tool with: `git clone https://github.com/pokemon-mini/c88-pokemini.git`
- Using Windows Explorer, go to the directory and right-click the file `install.ps1`, then select `Run with PowerShell`
- This will download all missing files. When asked if you want to configure environment variables, select YES

4. COMPILING THE MENU

You will see that the downloaded directory contains a folder called `examples`, which in turn contains a folder called `helloworld`. Navigate to this folder using the command prompt.

Run the command `mk88`. This will create the file `helloworld.min`, which can be run in a Pokémon Mini emulator, used in creating the single-ROM firmware, or used in the WebApp.



5. ADDITIONAL DOCUMENTATION

If you want to learn more, I recommend reading the following pages, where you will find much more information:

Pokemon-mini.net

Pokemon Mini Official Software Development Kit (Toolchain)

Mini Running Ant

FREQUENTLY ASKED QUESTIONS - FAQ

1. THE COMPUTER DOES NOT DETECT THE CARTRIDGE

You must press and hold the flash button while connecting the USB-C cable. Then you can release the button. This way, the computer will recognize the cartridge, and it will appear as a storage drive.

2. DO I LOSE MY SAVE DATA WHEN FLASHING NEW FIRMWARE?

The Pokémon Mini console does not store game saves on the cartridge itself, but rather in the console's internal memory.

Although the console's memory is limited, if you use many games that require saving, it is possible that some data could be overwritten.

3. CAN I CONNECT THE USB-C CABLE WHILE THE CARTRIDGE IS INSERTED IN THE CONSOLE?

Yes, there's no problem with that.

4. WHAT HAPPENS IF I PRESS THE FLASH BUTTON WHILE THE CARTRIDGE IS IN THE CONSOLE?

If you press and hold the button while turning on the console, the console will not detect the cartridge because the firmware does not start when the button is pressed.

If you press the button while the console is already on, nothing happens (the game is not interrupted).

5. WHAT ARE THE STATUS LEDS FOR?

The cartridge has two LEDs. The red one lights up only when the cartridge is connected via USB-C. The blue one can be controlled by the firmware at the user's discretion (accessible through GPIO16).

6. MY SHELL IS BROKEN, CAN I GET A REPLACEMENT?

We do not have replacement shells since we do not print them ourselves. If you want a new shell or wish to make one from another material/color, you can use the STL files from the repository to have it manufactured.

7. CAN I REPLACE THE SHELL WITH ONE FROM AN ORIGINAL GAME?

Yes, you can, although a few modifications are required:

- You'll need to cut the side tabs of the plastic so that both halves of the shell close only with the screws, without needing to slide one part into the other. This is essential because the USB-C port prevents that sliding.
- Additionally, if you want access to the USB-C connector, you'll also need to cut an opening in the plastic for it (and for the flash button).

8. WHEN I TURN ON THE CONSOLE, IT FREEZES

This happens because original games are designed to let you continue playing from where you left off when restarting the console. Since the console tries to load the previous game instead of the menu, it freezes when it can't find the expected data.

To prevent this, the games must be patched before being written into the firmware.

If you don't patch the games, you'll need to disconnect and reconnect the cartridge after playing an original game (homebrew titles don't cause this issue).

Unfortunately, I don't know the exact byte that needs to be modified or patched. If I did, I could probably update the Web Rom Patcher to handle it automatically during firmware generation.

Here's some additional information if you'd like to investigate it yourself. If you find anything, feel free to share it, I can try to fix it.

https://discord.com/channels/549770771963314216/573135821729824773/1400214331341279264